Міністерство освіти і науки України
Центральноукраїнський національний технічний університет
Механіко-технологічний факультет
Кафедра кібербезпеки та програмного забезпечення
Дисципліна: Базові методології та технології програмування

Лабораторна робота №10

Тема: «Реалізація програмних модулів оброблення даних складових типів з файловим введенням/виведенням»

Виконав: ст. гр. КН-25
Грицюк Є.В.
Перевірив: викладач
Коваленко А.С.

Кропивницький 2026

ТЕМА: Реалізація програмних модулів оброблення даних складових типів з файловим введенням/виведенням

МЕТА: полягає у набутті ґрунтовних вмінь і практичних навичок реалізації у Code::Blocks IDE мовою програмування C++ програмних модулів створення й оброблення даних типів масив, структура, об'єднання, множина, перелік, перетворення типів даних, використання файлових потоків та функцій стандартних бібліотек для оброблення символьної інформації.

ЗАВДАННЯ: 1. Реалізувати програмні модулі розв'язування задач 10.1–10.3 як складові статичної бібліотеки libModulesПрізвище.a (проект ModulesПрізвище лабораторних робіт №8–9). 2. Реалізувати тестовий драйвер автоматизованої перевірки програмних модулів розв'язування задач 10.1–10.3.

Варіант 8

Аналіз і постановка задачі 10.1

Умова: Прочитати речення з вхідного текстового файлу. У вихідний текстовий файл записати: авторську інформацію; змінене речення, де літери «к», «т», «У», «л», «Й» замінено на «<?>»; та частину вірша В. Симоненка (якщо кількість літер у вхідному реченні парна — "Матері", інакше — "Вклонися їй"). Якщо вихідний файл вже існує, його вміст має бути знищений (перезаписаний).

Вхідні дані: текстовий файл (ім'я передається як параметр).

Вихідні дані: текстовий файл (ім'я передається як параметр), що містить відформатовані текстові рядки.

Обмеження (ОДЗ): імена файлів повинні бути валідними; перед читанням/записом обов'язкова перевірка відкриття потоку. При підрахунку літер для вибору вірша знаки пунктуації та пробіли ігноруються.

Аналіз і постановка задачі 10.2

Умова: Прочитати те ж саме речення з вхідного текстового файлу. У вихідний текстовий файл дописати (без знищення попереднього вмісту) символи, що знаходяться на непарних позиціях у реченні, а також поточну локальну дату й час комп'ютера.

Вхідні дані: текстовий файл з оригінальним реченням.

Вихідні дані: текстовий рядок із символами та рядок із датою/часом, які дописуються у кінець існуючого вихідного файлу.

Обмеження (ОДЗ): вихідний файл має відкриватися в режимі дозапису (append).

Аналіз і постановка задачі 10.3

Умова: Отримати числові значення x, y, z та натуральне число b. У вихідний текстовий файл дописати результати виконання функції s_calculation(x, y, z) та подання числа b у двійковому форматі, сформувавши повноцінні україномовні речення.

Вхідні дані: дійсні числа x, y, z, ціле натуральне число b (від b > 0); ім'я вихідного файлу.

Вихідні дані: україномовні текстові повідомлення з результатами, що записуються у вихідний файл.

Обмеження (ОДЗ): значення x, y, z мають відповідати ОДЗ функції s_calculation; b має бути додатним. Режим відкриття файлу — дозапис.

Аналіз вимог, проектування архітектури та детальне проектування програмного модуля.

1. Функціональні вимоги:
    - Програмне забезпечення повинно зчитувати текст із файлів за допомогою std::ifstream та записувати результати за допомогою std::ofstream.

    - Імена файлів мають передаватися у функції як параметри (рядки типу std::string).
    - Модуль 10.1 має використовувати режим відкриття ios::out | ios::trunc (перезапис), а модулі 10.2 та 10.3 — ios::app (дозапис).
    - Усі вихідні повідомлення у файлі мають бути сформовані у вигляді зв'язних речень українською мовою.
    - Використання бібліотеки <ctime> для отримання поточного часу.
2. Нефункціональні вимоги:
    - Обов'язкова перевірка стану файлового потоку (is_open()). Якщо файл не відкрито, функція повинна безпечно завершити роботу або повернути код помилки.
    - Усі файлові потоки повинні бути явно закриті (close()) після завершення операцій.
    - Нові функції мають бути додані до існуючої статичної бібліотеки (ModulesHrytsiuk).

Проектування архітектури:

Нові функції розв'язання задач 10.1–10.3 розширять існуючий інтерфейс статичної бібліотеки у файлі ModulesHrytsiuk.h. Для забезпечення гнучкості та відповідності вимогам ("імена файлів є параметрами"), функції прийматимуть шляхи до файлів як аргументи.

Визначено такі нові прототипи функцій:

1. void task10_1_file_write(const std::string& in_filename, const std::string& out_filename);
2. void task10_2_file_append(const std::string& in_filename, const std::string& out_filename);
3. void task10_3_math_append(const std::string& out_filename, double x, double y, double z, int b);

Детальне проектування архітектури:

Модуль 1 (Задача 10.1): Перезапис файлу та обробка тексту

Алгоритм:

1. Відкрити потік ifstream для файлу in_filename. Перевірити відкриття.
2. Зчитати весь вміст файлу у рядкову змінну sentence. Закрити вхідний потік.
3. Відкрити потік ofstream для файлу out_filename у стандартному режимі (знищення старого вмісту). Перевірити відкриття.
4. Записати авторську інформацію: "Розробник: Грицюк Євгеній, ЦНТУ, м. Кропивницький, Україна, 2026 рік."
5. Створити копію sentence. Пройти циклом по символах; якщо символ збігається з 'к', 'т', 'У', 'л' або 'Й', замінити його на рядок "<?>". Записати змінене речення у файл.
6. Підрахувати кількість літер в оригінальному sentence (ігноруючи пробіли та розділові знаки).
7. Якщо лічильник парний (count % 2 == 0), записати у файл відповідний уривок вірша "Матері". Інакше — уривок "Вклонися їй".
8. Закрити вихідний потік.

Модуль 2 (Задача 10.2): Дозапис файлу та робота з датою

Алгоритм:

1. Відкрити ifstream для in_filename. Зчитати рядок sentence. Закрити потік.
2. Відкрити ofstream для out_filename у режимі дозапису (std::ios::app).
3. Записати інформаційне повідомлення: "Непарні символи речення: ".
4. Пройти циклом по рядку sentence з кроком 2 (вибираючи символи на непарних позиціях, індекси 0, 2, 4...). Записати ці символи у файл.
5. Отримати поточний час: time_t now = time(0); та перетворити його на рядок за допомогою ctime(&now).
6. Записати у файл повідомлення: "Дата й час дозапису: " та згенерований рядок часу.
7. Закрити файловий потік.

Модуль 3 (Задача 10.3): Дозапис результатів обчислень

Алгоритм:

1. Відкрити ofstream для out_filename у режимі дозапису (std::ios::app). Перевірити успішність відкриття.
2. Викликати існуючу функцію s_calculation(x, y, z). Зберегти результат.
3. Записати у файл відформатоване речення: "Результат виконання функції s_calculation з аргументами x=..., y=..., z=... дорівнює: ..."
4. Ініціалізувати порожній рядок (або використати std::bitset) для формування двійкового коду числа b.
5. Записати у файл повідомлення: "Число ... у двійковому коді: ..." та вивести отриманий двійковий формат.
6. Закрити файловий потік.

Artifact: Test Suite

Date: DATE \@ "M/d/yyyy"
10/22/2024

Назва тестового набору Test Suite Description	TS_10_1
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_10_1
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	1. Створити in_1.txt із текстом: Привіт, світ! (10 літер — парне число).	Створено out_1.txt. Вміст: Розробник: Грицюк Євгеній, ЦНТУ, м. Кропивницький, Україна, 2026.	passed

	2. Вихідний файл out_1.txt відсутній. discount_uah = 0.0 final_amount = 0.0	Приві<?>, сві<?>! В хаті сонячній промінь косо На долівку ляга з вікна... Твої чорні шовкові коси Припорошила вже сивина.	
TC-02	1. Створити in_2.txt із текстом: Успіх (5 літер — непарне число). 2. Створити out_2.txt із текстом Старий текст.	Файл out_2.txt перезаписано (старий текст зник). Вміст: Розробник: Грицюк Євгеній, ЦНТУ... <?>спіх Коли малим ти вперше став на ноги - Яка ж то радість матері була! Від тихої колиски до порога Вона тебе за руку провела.	passed

Artifact: Test Suite

Date: DATE \@ "M/d/yyyy"
10/22/2024

Назва тестового набору Test Suite Description	TS_10_2
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_10_2
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	1. Створити in_3.txt із текстом: АБВГД. (Символи на непарних позиціях: А, В, Д). 2. Створити out_3.txt із текстом: Початок.	Вміст out_3.txt (старий текст зберігся): Початок. Непарні символи речення: АВД	passed

		Дата й час дозапису: Thu Apr 16 15:16:53 2026	
TC-02	1. Створити in_4.txt із текстом: 123456. 2. Вихідний файл out_4.txt відсутній.	Створено out_4.txt. Вміст: Непарні символи речення: 135 Дата й час дозапису: [Поточний час системи]	passed

Назва тестового набору Test Suite Description	TS_10_3
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_10_3
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	1. Створити порожній вихідний файл out_5.txt.	Вміст out_5.txt: Результат виконання функції s_calculation з аргументами x=1, y=2, z=3 дорівнює: 29.719. Число 5 у двійковому коді: 101.	passed
TC-02	1. Файл out_5.txt зберігся з попереднього тесту.	Вміст out_5.txt доповнюється: [...старий текст з TC-01...] Результат виконання функції s_calculation з аргументами x=0, y=0, z=0 дорівнює: 0. Число 10 у двійковому коді: 1010.	passed

Лістинг програми

```
#include <cmath>

#include <stdexcept>

#include <string>

#include <fstream>
```

```
#include <ctime>

#include <vector>


constexpr double PI = 3.14159265358979323846;


double s_calculation(double x, double y, double z) {

    if (z < 0.0) {

        throw std::invalid_argument("z не може бути від'ємним");

    }

    double term1 = pow(x + y + z, 2.0);

    double term2 = 2.0 * PI * sqrt(z / 3.0);

    return term1 - term2;

}


int task9_1_discount(double purchase_amount, double &discount_uah, double
&final_amount) {

    if (purchase_amount < 0) {

        discount_uah = 0;

        final_amount = 0;

        return 0;

    }


    int percent = 0;

    if (purchase_amount < 100) {

        percent = 5;

    } else if (purchase_amount < 200) {

        percent = 7;

    } else if (purchase_amount < 300) {
```

```
        percent = 9;
    } else if (purchase_amount < 400) {
        percent = 12;
    } else if (purchase_amount < 500) {
        percent = 15;
    } else if (purchase_amount <= 1000) {
        percent = 20;
    } else {
        percent = 25;
    }

    discount_uah = (purchase_amount * percent) / 100.0;
    final_amount = purchase_amount - discount_uah;

    return percent;
}



std::string task9_2_size_converter(int ukr_size) {
    switch (ukr_size) {
        case 44: return "Міжнародний: S, UK: 34, Франція: 2";
        case 46: return "Міжнародний: M, UK: 36, Франція: 3";
        case 48: return "Міжнародний: L, UK: 38, Франція: 4";
        case 50: return "Міжнародний: XL, UK: 40, Франція: 5";
        case 52: return "Міжнародний: XXL, UK: 42, Франція: 6";
        case 54: return "Міжнародний: XXXL, UK: 44, Франція: -";
        default: return "Розмір не знайдено";
    }
```

```
}



int task9_3_bit_counter(int N) {

    if (N < 0) return 0;


    bool d3_is_one = (N & (1 << 3)) != 0;


    int count = 0;

    int temp = N;


    while (temp > 0) {

        int current_bit = temp & 1;


        count += d3_is_one ? (current_bit == 0 ? 1 : 0) : (current_bit == 1 ?
1 : 0);


        temp >>= 1;

    }


    return count;

}


void task10_1_file_write(const std::string& in_filename, const std::string&
out_filename) {

    using namespace std;


    ifstream in_file(in_filename);

    if (!in_file.is_open()) return;
```

```
    string sentence;
    getline(in_file, sentence);
    in_file.close();

    string modified_sentence = sentence;
    string targets[] = {"к", "т", "У", "л", "Й"};
    for (const string& target : targets) {
        size_t pos = 0;
        while ((pos = modified_sentence.find(target, pos)) != string::npos) {
            modified_sentence.replace(pos, target.length(), "<?>");
            pos += 3;
        }
    }

    int letter_count = 0;
    for (char c : sentence) {
        if (c != ' ' && c != '.' && c != ',' && c != '!' && c != '?' && c !=
'-' && c != ';' && c != ':') {
            letter_count++;
        }
    }

    ofstream out_file(out_filename, ios::out | ios::trunc);
    if (!out_file.is_open()) return;

    out_file << "Розробник: Грицюк Євгеній, КН-25\n";
    out_file << "Установа: ЦНТУ, м. Кропивницький, Україна, 2026 рік\n\n";
```

```
    out_file << modified_sentence << "\n\n";


    if (letter_count % 2 == 0) {

        out_file << "В хаті сонячній промінь косо\nНа долівку ляга з
вікна...\n";

        out_file << "Твої чорні шовкові коси\nПрипорошила вже сивина.\n";

    } else {

        out_file << "Коли малим ти вперше став на ноги -\nЯка ж то радість
матері була!\n";

        out_file << "Від тихої колиски до порога\nВона тебе за руку
провела.\n";

    }


    out_file.close();

}


void task10_2_file_append(const std::string& in_filename, const std::string&
out_filename) {

    using namespace std;


    ifstream in_file(in_filename);

    if (!in_file.is_open()) return;


    string sentence;

    getline(in_file, sentence);

    in_file.close();


    ofstream out_file(out_filename, ios::app);

    if (!out_file.is_open()) return;
```

```
    out_file << "\n--- Задача 10.2 ---\n";

    out_file << "Непарні символи речення: ";


    for (size_t i = 0; i < sentence.length(); i += 2) {

        out_file << sentence[i];

    }

    out_file << "\n";


    time_t now = time(0);

    string time_str = ctime(&now);

    out_file << "Дата й час дозапису: " << time_str;


    out_file.close();

}


void task10_3_math_append(const std::string& out_filename, double x, double
y, double z, int b) {

    using namespace std;


    ofstream out_file(out_filename, ios::app);

    if (!out_file.is_open()) return;


    double s_result = s_calculation(x, y, z);


    out_file << "\n--- Задача 10.3 ---\n";

    out_file << "Результат виконання функції s_calculation з аргументами ";

    out_file << "x=" << x << ", y=" << y << ", z=" << z << " дорівнює: " <<
s_result << ".\n";
```

```
    string bin_str = "";
    int temp = b;
    if (temp == 0) {
        bin_str = "0";
    } else {
        while (temp > 0) {
            bin_str = (temp % 2 == 0 ? "0" : "1") + bin_str;
            temp /= 2;
        }
    }

    out_file << "Число " << b << " у двійковому коді: " << bin_str << ".\n";

    out_file.close();
}
```

Лістинг TestDriver

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
#include <Windows.h>

#include "ModulesHrytsiuk.h"

using namespace std;

string readFileContent(const string& filename) {
```

```
    ifstream file(filename);

    if (!file.is_open()) return "";

    string content((istreambuf_iterator<char>(file)),
istreambuf_iterator<char>());

    file.close();

    return content;

}



void logResult(ofstream& protocol, const string& tc_id, const string&
input_args, bool passed) {

    string status = passed ? "[PASSED]" : "[FAILED]";



    cout << left << setw(12) << tc_id << " | " << setw(20) << input_args << "
| " << status << endl;



    protocol << left << setw(12) << tc_id << " | " << setw(20) << input_args
<< " | " << status << "\n";

}



int main() {

    SetConsoleOutputCP(1251);

    SetConsoleCP(1251);



    ofstream protocol("test_protocol.txt", ios::out | ios::trunc);

    if (!protocol.is_open()) {

        cout << "Помилка: не вдалося створити файл протоколу!" << endl;

        return 1;

    }



    protocol << "=== ПРОТОКОЛ АВТОМАТИЗОВАНОГО ТЕСТУВАННЯ (ЛР 10) ===\n";
```

```
    protocol << "Розробник: Грицюк Євгеній, КН-25\n\n";

    cout << "ЗАПУСК АВТОМАТИЗОВАНИХ ТЕСТІВ...\n";
    cout << string(50, '-') << endl;

    protocol << "--- Задача 10.1 ---\n";
    cout << "--- Задача 10.1 ---\n";

    struct TC10_1 { string id, in_f, out_f, text, expected; };
    const TC10_1 tests10_1[] = {
        {"TC-10.1-01", "in_1.txt", "out_1.txt", "Привіт, світ!", "Приві<?>,
сві<?>!"},
        {"TC-10.1-02", "in_2.txt", "out_2.txt", "Успіх", "<?>спіх"}
    };

    for (const auto& tc : tests10_1) {
        ofstream in_file(tc.in_f);
        in_file << tc.text;
        in_file.close();

        task10_1_file_write(tc.in_f, tc.out_f);

        string result_content = readFileContent(tc.out_f);
        bool passed = (result_content.find(tc.expected) != string::npos);

        logResult(protocol, tc.id, "File: " + tc.in_f, passed);
    }
```

```
    protocol << "\n--- Задача 10.2 ---\n";
    cout << "\n--- Задача 10.2 ---\n";

    struct TC10_2 { string id, in_f, out_f, text, expected; };
    const TC10_2 tests10_2[] = {
        {"TC-10.2-01", "in_3.txt", "out_3.txt", "АБВГД", "АВД"},
        {"TC-10.2-02", "in_4.txt", "out_4.txt", "123456", "135"}
    };

    for (const auto& tc : tests10_2) {
        ofstream in_file(tc.in_f);
        in_file << tc.text;
        in_file.close();

        task10_2_file_append(tc.in_f, tc.out_f);

        string result_content = readFileContent(tc.out_f);
        bool passed = (result_content.find(tc.expected) != string::npos);

        logResult(protocol, tc.id, "File: " + tc.in_f, passed);
    }

    protocol << "\n--- Задача 10.3 ---\n";
    cout << "\n--- Задача 10.3 ---\n";

    struct TC10_3 { string id, out_f; double x, y, z; int b; string exp_bin;
};
    const TC10_3 tests10_3[] = {
```

```
        {"TC-10.3-01", "out_5.txt", 1.0, 2.0, 3.0, 5, "101"},
        {"TC-10.3-02", "out_5.txt", 0.0, 0.0, 0.0, 10, "1010"}
    };

    ofstream clean_out("out_5.txt", ios::trunc); clean_out.close();

    for (const auto& tc : tests10_3) {
        task10_3_math_append(tc.out_f, tc.x, tc.y, tc.z, tc.b);

        string result_content = readFileContent(tc.out_f);
        bool passed = (result_content.find(tc.exp_bin) != string::npos);

        string args = "x,y,z | b=" + to_string(tc.b);
        logResult(protocol, tc.id, args, passed);
    }

    protocol.close();
    cout << string(50, '-') << endl;
    cout << "Тестування завершено. Результати записано у файл
test_protocol.txt\n";

    system("pause");
    return 0;
}
```

```
ЗАПУСК АВТОМАТИЗОВАНИХ ТЕСТІВ...
--------------------------------------------------
--- Задача 10.1 ---
TC-10.1-01   | File: in_1.txt       | [PASSED]
TC-10.1-02   | File: in_2.txt       | [PASSED]

--- Задача 10.2 ---
TC-10.2-01   | File: in_3.txt       | [PASSED]
TC-10.2-02   | File: in_4.txt       | [PASSED]

--- Задача 10.3 ---
TC-10.3-01   | x,y,z | b=5          | [PASSED]
TC-10.3-02   | x,y,z | b=10         | [PASSED]
--------------------------------------------------
Тестування завершено. Результати записано у файл test_protocol.txt
```

Назва тестового набору Test Suite Description	TS_10_3
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_10_3
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	in_1.txt: "Привіт, світ!" (парна кількість літер).	Авторські дані (Грицюк Є., КН-25); текст: "Приві<?>, сві<?>!"; вірш "Матері".	passed
TC-02	in_2.txt: "Успіх" (непарна кількість літер).	Авторські дані; текст: "<?>спіх"; вірш "Вклонися їй".	passed
TC-03	in_3_ext.txt: "Кіт і кішка."	Авторські дані; текст: "<?>і<?> і <?>іш<?>а."; вибір вірша згідно з кількістю літер.	passed

Назва тестового набору Test Suite Description	TS_10_3
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_10_3
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	in_3.txt: "АБВГД"	Рядок: "Непарні символи речення: АВД"; актуальна дата й час.	passed
TC-02	in_4.txt: "123456"	Рядок: "Непарні символи речення: 135"; актуальна дата й час..	passed
TC-03	in_5_ext.txt: "Програмування"	Рядок: "Непарні символи речення: Пормвня"; актуальна дата й час.	passed

Назва тестового набору Test Suite Description	TS_10_3
Назва проекта / ПЗ Name of Project / Software	Hrytsiuk-task_10_3
Рівень тестуванн Level of Testin	Модульне тестування
Автор тестсьют Test Suite Autho	Грицюк Євгеній
Виконавец Implemente	Грицюк Євгеній

Ід-р тест-кейса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result

TC-01	x=1.0, y=2.0, z=3.0, b=5	Результат s_calculation; речення: "Число 5 у двійковому коді: 101."	passed
TC-02	x=0, y=0, z=0, b=10	Результат s_calculation; речення: "Число 10 у двійковому коді: 1010."	passed
TC-03	x=2.0, y=2.0, z=2.0, b=255	Результат s_calculation; речення: "Число 255 у двійковому коді: 11111111."	passed

Аргументи

1. Опанування принципу енергонезалежного зберігання даних у зовнішніх текстових файлах
2. Практичне використання класів ifstream та ofstream з бібліотеки fstream
3. Знайомство з режимами відкриття файлів, зокрема з режимом безпечного дозапису ios::app
4. Розуміння режиму повного перезапису та очищення старого вмісту файлу ios::trunc
5. Формування обов'язкової звички перевіряти успішність відкриття файлового потоку is_open
6. Практика коректного закриття файлових потоків close для уникнення витоку пам'яті
7. Знайомство із системною бібліотекою ctime для взаємодії з годинником комп'ютера
8. Здобуття досвіду отримання локального системного часу в момент виконання коду
9. Вміння перетворювати системний час у зручний для читання текстовий формат за допомогою функції ctime
10. Опанування концепції повністю автоматизованого тестування без використання сторонніх фреймворків
11. Розуміння сучасного підходу тестування на основі даних
12. Практика використання структур для логічної організації даних тест-кейсу
13. Вміння створювати та ініціалізувати константні масиви структур для

зберігання тестових сценаріїв
14. Навичка програмного порівняння очікуваного та фактичного результатів
15. Створення власної системи протоколювання процесу тестування у зовнішній файл
16. Засвоєння фундаментального факту: текст у комп'ютері це масив байтів, а не просто літери
17. Глибоке розуміння проблеми конфлікту кодувань UTF-8 проти CP1251 або ANSI
18. Навичка виявлення та виправлення неправильного відображення кирилиці у текстових файлах
19. Вміння синхронізувати кодування у різних файлах вихідного коду одного проекту
20. Опанування методу зчитування всього вмісту файлу в один рядок за допомогою ітераторів
21. Практика використання методу find для перевірки наявності потрібного підрядка
22. Навичка безпечної заміни частини тексту за допомогою методу replace
23. Розуміння проблеми нескінченного циклу під час пошуку та заміни, вміння правильно зсувати індекс
24. Досвід лексичного аналізу тексту з ігноруванням знаків пунктуації та пробілів
25. Вміння передавати імена файлів у функції як константні посилання
26. Усвідомлення головної переваги автоматизації: можливість виконати десятки тестів за мілісекунди
27. Практичне застосування математичного алгоритму переведення десяткового числа у двійкове
28. Навичка форматування виводу у файл, який працює аналогічно до виводу в консоль
29. Досвід одночасної роботи з багатопроектним робочим простором у середовищі Code Blocks
30. Закріплення навичок налаштування шляхів пошуку заголовкових файлів для компілятора
31. Практика налаштування лінкера для підключення оновленої статичної бібліотеки
32. Використання сучасного синтаксису циклу for для обходу масивів

33. Вміння динамічно генерувати необхідні вхідні файли прямо під час роботи програми
34. Розвиток навичок відлагодження через візуальний аналіз вмісту пошкоджених вихідних файлів
35. Розуміння того, що успішна компіляція не гарантує логічно правильної роботи алгоритму
36. Використання маніпуляторів простору iomanip для створення рівних таблиць у логах
37. Формування звички перевіряти стан і наявність файлу перед його обробкою
38. Практика безпечного посимвольного доступу до елементів рядка за їхнім індексом
39. Використання кроку циклу для швидкої вибірки символів на непарних позиціях
40. Використання оператора остачі від ділення для перевірки парності кількості літер
41. Досвід успішної інтеграції старого математичного коду в нову архітектуру
42. Розуміння принципу чорного ящика під час тестування: важливі лише вхідні дані та результат
43. Вміння використовувати звичайний текстовий редактор як інструмент для перевірки програм
44. Практика паралельного виводу інформації: дублювання даних на екран та у файл
45. Створення самодокументованого коду з інтуїтивно зрозумілими статусами виконання
46. Вміння очищати тестове середовище: видалення застарілих файлів перед новим запуском
47. Розуміння важливості повної незалежності тест-кейсів один від одного
48. Вміння формувати надійні докази працездатності програми для здачі роботи
49. Закріплення розуміння повного циклу розробки: від проектування до тестування
50. Отримання професійної інженерної впевненості завдяки успішному вирішенню складних багів

Висновок: Під час виконання лабораторної роботи було успішно закріплено навички роботи з текстовими файловими потоками (<fstream>) мовою C++. Головним практичним результатом стало створення повністю автоматизованого тест-драйвера для перевірки розроблених раніше модулів статичної бібліотеки. У процесі відлагодження тест-кейсів було на практиці досліджено та успішно вирішено проблему несумісності текстових кодувань (UTF-8 та ANSI/Windows-1251). Здобутий досвід розробки наборів тестових даних, автоматичного порівняння результатів та ведення лог-файлів є фундаментальним кроком до створення надійного та професійного програмного забезпечення.